

EduGenie: Google Gemini Powered Learning Assistant

Important Contents – 10–15 Page Project Report

Project Theme: AI-powered personalized learning assistant

Technologies: Google Gemini API, FastAPI, Python, HTML, CSS, Jinja2 and Uvicorn

Submitted by: Tella Divya Sree

Mentor: Siri

Date: 11/04/2025

Project Overview

EduGenie is a lightweight AI-powered educational assistant designed to make learning simpler, faster and more interactive. It combines cloud-based generative AI with a simple web interface so learners can ask questions, understand difficult concepts, generate quizzes, summarize passages and obtain structured learning recommendations.

1. Project Description

EduGenie supports students and self-learners through a collection of educational AI functions. The system accepts natural-language input and routes it to the appropriate module. The response is then displayed through a simple HTML and CSS interface.

Major Capabilities

- Question answering – provides concise answers to academic and general questions.
- Concept explanation – converts difficult concepts into simpler language.
- Quiz generation – creates multiple-choice questions from supplied content.
- Summarization – reduces long passages into concise revision material.
- Learning recommendations – creates a structured beginner-to-advanced learning path.

Objectives

- Make educational information easier to understand.
- Provide quick AI-assisted academic support.
- Reduce the effort required to create revision material.
- Personalize learning recommendations according to a learner's topic and level.

Key Scenarios

- A student asks which is the largest ocean.
- A learner generates a quiz to check understanding of the Pythagoras theorem.
- A learner studying SQL requests a structured beginner-to-advanced learning path.

2. System Architecture

The architecture separates the user interface, FastAPI backend and AI-powered modules. The frontend collects the learner's request, the backend identifies the selected task and the corresponding module communicates with the appropriate AI model.

Student / Learner	HTML + CSS Frontend	FastAPI Backend	AI Modules	Response
-------------------	------------------------	-----------------	------------	----------

Architectural Components

- Frontend: task selection, input area, submit action and result display.
- FastAPI: exposes endpoints and connects HTTP requests with Python functions.
- Explanation module: provides simplified explanations.
- Q&A; module: handles academic and general questions.
- Quiz module: generates structured MCQs.
- Summary module: produces concise summaries.
- Learning path module: generates personalized recommendations.

Data Flow

Input	Task Selection	API Endpoint	Model Processing	Formatted Output
-------	----------------	--------------	------------------	------------------

3. Pre-requisites and Technologies

Python 3.10+

Python provides the main programming environment for the backend and AI integration. Installation can be verified from a terminal with the Python version command.

FastAPI Framework

FastAPI is used to build the backend API. It provides a lightweight way to define endpoints and connect HTTP requests with Python functions.

Google Gemini API

The Gemini API supplies cloud-based generative AI capabilities for question answering, quiz generation, summarization and learning recommendations. API keys should be stored securely.

Uvicorn

Uvicorn is the ASGI server used to run the FastAPI application locally.

Jinja2, HTML and CSS

Jinja2 supports HTML templating, while HTML and CSS provide the basic web interface and responsive styling.

Typical setup commands

```
pip install uvicorn  
pip install jinja2
```

4. Milestone 1 – Model Selection and Folder Architecture

The project combines cloud-based intelligence with lightweight local processing. The supplied project design identifies Gemini 1.5 Pro for major generative tasks and LaMini-Flan-T5-783M for concept explanation.

Model	Use	Purpose
Gemini 1.5 Pro	Q&A;, summary, quiz, learning path	Cloud-based generative intelligence
LaMini-Flan-T5-783M	Concept explanation	Lightweight instruction-tuned local processing

Folder Architecture

EduGenie/

- main.py
- explanation_module.py
- qna.py
- quiz_module.py
- summary_module.py
- learning_path.py
- templates/index.html
- static/style.css
- requirements.txt

This modular organization makes individual functions easier to develop, test and maintain.

5. Milestone 2 – Explanation and Q&A; Modules

Explanation Module

The explanation module is designed to simplify complex educational topics. LaMini-Flan-T5 is described in the project as a lightweight instruction-tuned model for concise and readable explanations, making it useful for beginners and learners with limited background knowledge.

Topic / Question	Explanation Module	Language Model	Simple Explanation
------------------	--------------------	----------------	--------------------

Q&A; Module

The Q&A; module uses Gemini to respond to general knowledge and academic questions. The model receives the learner's question and generates a contextual response for display in the web interface.

User Question	POST /qa	Gemini	Answer
---------------	----------	--------	--------

Example

Input: Which is the largest ocean?

Purpose: Demonstrates a simple factual question-answering interaction.

Educational Value

- Provides immediate assistance.
- Allows natural-language questions.
- Supports diverse academic topics.

6. Quiz Module

The quiz module converts a supplied passage or topic into multiple-choice questions. The project design specifies three questions, each with four options and a correct answer.

Quiz Generation Process

Passage / Topic	Structured Prompt	Gemini	JSON Response	Quiz Display
-----------------	-------------------	--------	---------------	--------------

Important Processing Steps

- The prompt instructs the model to generate relevant questions.
- Each question contains four answer options.
- A correct answer is included for evaluation.
- Markdown code fences are cleaned before parsing.
- The cleaned response is parsed into a Python structure.
- Generation or parsing errors are handled with an error message.

Learning Benefit

Quizzes support active recall. Instead of only reading an explanation, learners can test their understanding of the supplied material.

Example Scenario

A student studying the Pythagoras theorem selects “Generate Quiz”. EduGenie creates MCQs from the relevant content and gives feedback based on the selected answer.

7. Summary and Learning Path Modules

Summary Module

The summary module uses generative AI to convert long educational passages into shorter, easier-to-review content. The goal is to retain central ideas while reducing repetition.

Long Passage	Summarization Prompt	Gemini	Concise Summary
--------------	----------------------	--------	-----------------

Advantages

- Useful for quick revision.
- Reduces the amount of text to review.
- Helps identify key ideas.
- Makes lengthy material easier to approach.

Learning Path Module

The learning path module generates a structured plan for a requested topic. It can organize beginner, intermediate and advanced concepts, suggest resources and provide step-by-step guidance appropriate to the learner's level.

Topic + Level	Recommendation Prompt	Gemini	Structured Learning Path
---------------	-----------------------	--------	--------------------------

Typical learning-path elements

- Prerequisites and beginner concepts.
- Intermediate and advanced topics.
- Suggested sequence of study.
- Videos, articles or books as possible resources.
- Step-by-step progression.

8. FastAPI Backend and API Endpoints

FastAPI acts as the central backend layer. Each major EduGenie function is exposed through a dedicated endpoint. The frontend sends a request to the relevant endpoint, and the backend returns the generated result.

Endpoint	Function
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Learning recommendations

Backend Flow

HTTP POST Request	FastAPI Endpoint	Module Logic	AI Model	HTTP Response
-------------------	------------------	--------------	----------	---------------

Local Execution

The application can be started locally with Uvicorn using `uvicorn main:app --reload`. The local interface can then be opened in a browser.

9. Frontend Development and Live Integration

The frontend is intentionally simple so learners can focus on the educational task rather than a complicated software interface.

HTML Interface

- Task dropdown: Explain, QnA, Quiz, Summary and Recommend Path.
- Text area for a question, topic or passage.
- Submit button to send the request.
- Result container for displaying generated output.

CSS Styling

- Responsive page structure.
- Styled buttons and input areas.
- Readable result presentation.

Live Integration

When the learner submits the form, the frontend sends a POST request to the FastAPI backend. The backend processes the selected task and returns the result, which is displayed below the input area.

Learner Input	HTML Form	POST Request	FastAPI	AI Processing	Result
---------------	-----------	--------------	---------	---------------	--------

User Experience

The combination of a simple interface and modular backend lets learners move between different study activities without learning a complicated software system.

10. Deployment, Testing and Exploration

Local Deployment

- Install the required Python packages.
- Configure the Gemini API key securely.
- Start the FastAPI application using Uvicorn.
- Open the local web interface in a browser.

Functional Testing

- Ask a factual or academic question and verify the response.
- Request an explanation of a difficult topic.
- Generate a quiz from a passage.
- Summarize a long paragraph.
- Generate a learning recommendation path.

Exploring EduGenie

Feature	Expected interaction
Asking questions	Enter a question and receive an AI-generated answer.
Explanation	Enter a topic and receive a simplified explanation.
Summarization	Paste a long passage and obtain a concise version.
Quiz	Generate three MCQs with four options each.
Learning recommendations	Receive a stepwise beginner-to-advanced plan.

11. Conclusion and Future Enhancements

EduGenie demonstrates how generative AI can be combined with a lightweight web application to support personalized learning. Its modular design brings together question answering, concept explanation, quiz generation, summarization and learning recommendations in one educational assistant.

The development process also highlights practical challenges such as maintaining response quality, testing AI-generated content, improving performance, protecting API credentials and designing an interface that remains clear for different learners.

Future Enhancements

- Voice-based interaction for hands-free learning.
- Multilingual support for learners from different language backgrounds.
- Mobile application with offline-oriented features.
- Progress dashboards and learning analytics.
- Gamification using badges, streaks and challenges.
- Adaptive learning paths based on learner progress.
- Group study sessions and teacher/parent dashboards.
- Integration with Learning Management Systems such as Moodle or Google Classroom.
- Image and PDF input for doubt solving and summarization.
- Smart notifications and real-time synchronization.

Final Statement

EduGenie provides a foundation for an accessible and interactive learning companion. By combining generative AI, a modular backend and a simple user interface, the project shows how digital learning tools can support comprehension, revision, assessment and guided learning.

Submitted by: Tella Divya Sree **Mentor:** Siri **Date:** 11/04/2025